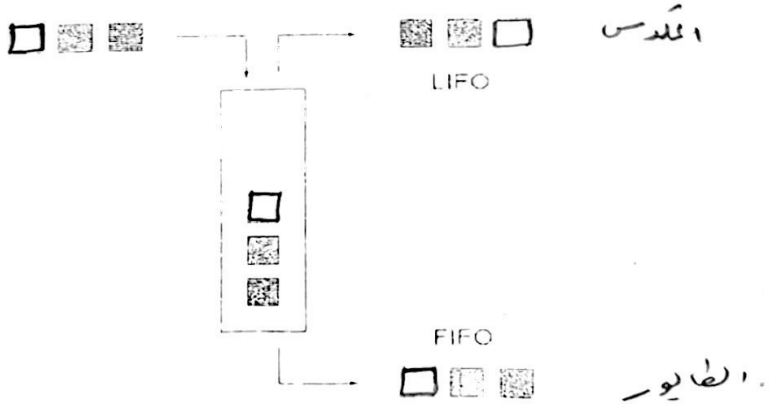
الفصل الثالث

الطابور Queue

3. 1 مقدمة:

الطابور عبارة عن نوع من هياكل البيانات الخطية ويشبه المكدس لتخزين المعلومات بشكل مؤقت مع فارق يكمن في إن التنظيم المتبع لإدخال البيانات واخراجها هو (First In First Out) ويختصر بكتابة الأحرف الأولى ليكون (FIFO)، أي إن الداخل أولا يكون هو الخارج أولا وتكون عملية الإضافة من النهاية والحذف من البداية وهذا يعني انه يوجد للطابور مؤشرين هما مؤشر الراس ويسمى (Head, Front) ومؤشر الذيل ويسمى (Tail, Rear)، وعند الإضافة فأننا نزيد من قيمة الذيل بواحد وعند الحذف فاننا نزيد من قيمة الرأس بواحد أيضا فتكون البيانات متراصة بشكل متتالي ومتقاربة على شكل خط وليست على مواقع متفرقة بالذاكرة أي هو أشبه بالطابور المدرسي أو طابور الانتظار في مستشفى، كما في الشكل (6).



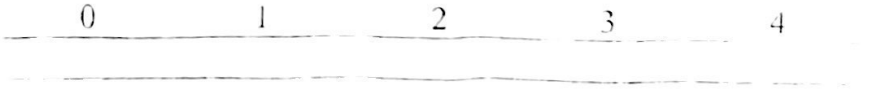
الشكل (6): تمثيل الطابور باستخدام المصفوفة .

يمكن تطبيق الطابور باستخدام مصفوفة ذات بعد واحد وبالسعة المرغوبة (size) ونوع البيانات المطلوب (Data Type) واستخدام متغير يسمى (Rear) يشير إلى موقع العنصر الأخير في الطابور والمتغير (Front) يستعمل كمؤشر يشير إلى موقع أول عنصر في الطابور.

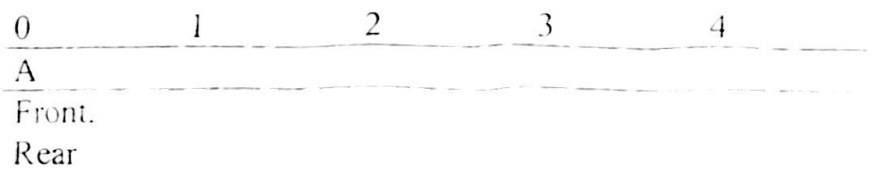
مثال (34): لنفرض إن لدينا طابور متكون من خمسة مواقع، قم بالعمليات الآتية:-

إن قيمة المؤشرين في الحالة الابتدائية يساوي (-1) عندما يكون الطابور خالي من العناصر.

Front=-1
Rear=-1



- إضافة (A)



عملية الإضافة تتم من المؤشر (Rear) ولذلك يصبح المؤشر يساوي صفر (أول موقع في الطابور)، وبما إن هذه أول عملية إضافة فان المؤشر (Front) يؤشر على الموقع صفر أيضاً.

Scanned with CamScanner

- إضافة (B)

0	1	2	3	4
A	B			
Front	Rear			

عملية الإضافة تتم من المؤشر (Rear) ولذلك يصبح المؤشر يساوي واحد، ويبقى المؤشر (Front) يؤشر على الموقع صفر (اول موقع في الطابور).

- حذف

0	1	2	3	4
	B			
	Front, Rear			

عملية الحذف تتم من المؤشر (Front) ولذلك يصبح المؤشر يساوي واحد، ويبقى المؤشر (Rear) يؤشر على الموقع الاول.

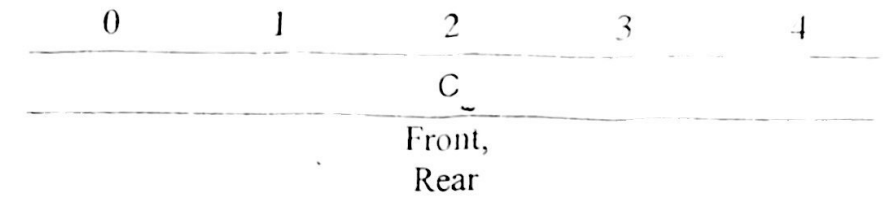
- إضافة (C):

0	1	2	3	4
	B	C		
	Front	Rear		

عملية الإضافة تتم من المؤشر (Rear) ولذلك يصبح المؤشر يساوي اثنان، ويبقى المؤشر (Front) يؤشر على الموقع الاول.

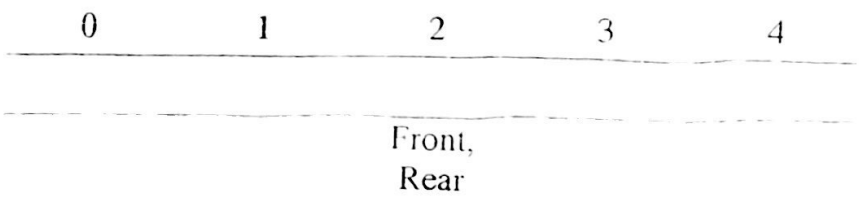
- حذف:

Scanned with CamScanner



عملية الحذف تتم من المؤشر (Front) ولذلك يصبح المؤشر يساوي اثنان، ويبقى المؤشر (Rear) يؤشر على الموقع الثاني.

- **حذف:**



عملية الحذف تتم من المؤشر (Front) وبما إن هذا العنصر هو أخر عنصر في الطابور والمؤشرين يؤشران على نفس الموقع فان هذا يعني أن الطابور خالي ويتوجب تصفير المؤشرين.

ويعرف الطابور برمجيا كالأتي:

```
const int size=3;
int Queue[size], front=-1, rear=-1;
```


Scanned with CamScanner

3. 2 عملية إضافة وحذف عنصر إلى الطابور

3. 2. 1 عملية إضافة عنصر (Insert) إلى الطابور

لتنفيذ عملية الإضافة بشكل صحيح نتبع الخطوات الآتية:

1. التحقق من ان الطابور غير مملوء (أي يحتوي على مكان للإضافة عنصر جديد) وان المؤشر (Rear ≠ Size-1) لتجنب حالة الفيضان (Overflow) وتعذر تنفيذ عملية الإضافة.
2. تحديث قيمة المؤشر ليشير إلى الموقع التالي (Rear=rear + 1).
3. إضافة العنصر الجديد في موقع المؤشر الجديد (Queue[Rear]).

3. 2. 2 عملية حذف عنصر (Delete) من الطابور

ان تنفيذ عملية حذف اي عنصر من الطابور يجب ان تكون وفق الخطوات الأتية:

1. التحقق من ان الطابور غير خال (أي يحتوي على عناصر لحذفها منه) وان المؤشر (Front ≠ -1) لتجنب حالة الفيضان (Underflow) وتعذر تنفيذ عملية الحذف.
2. سحب عنصر من الموقع الذي يشير اليه (Front).
3. تحديث قيمة المؤشر ليشير (Front=front+1) إلى الموقع التالي.

- **البرنامج الفرعي لعملية إضافة عنصر (Insert) إلى الطابور:**

```
void insert()
{
if(rear==size-1) cout<<"FULL QUEUE"<<endl;
else
{
rear=rear + 1;
```

```
cout<<"Enter the Element : ";
cin>>element;
Queue[rear]=element;
}
if (front==-1) front=0;
}
```

هذا البرنامج الفرعي يضيف عنصر (Element) واحد إلى الطابور ويمكن استدعائه في البرنامج الرئيسي باي عدد من المرات باستخدام احد ايعازات التكرار والذي يتضمن استدعاء البرنامج الفرعي (Insert) واضافة عنصر (Element) جديد إلى المكدس. ان هذه الصيغة تسمح باستدعائه في البرنامج الرئيسي في اكثر من موقع ولعدة مرات.

- البرنامج الفرعي لعملية حذف عنصر (Delete) من الطابور:

```
voiddelet()
{
if (front==-1) cout<<"EMPTY Queue "<<endl;
elsecout<<Queue[front]<<endl;
if(rear==front) { front=-1; rear=-1;}
else front=front +1;
}
```

هذا البرنامج الفرعي ياخذ العنصر (Element) الذي يشير اليه (Front) ويعرضه على الشاشة. ولغرض سحب اكثر من عنصر من الطابور بصورة متتابعة فان هذا البرنامج يستدعى باي عدد من المرات وفي اي موقع في البرنامج الرئيسي.

- البرنامج الفرعي لطباعة (Print) محتويات الطابور:

```
void print()
{
for (int i=front; i<rear+1; ++i)
```


Scanned with CamScanner

```
cout<<Queue[i]<<endl;
}
```

يطبع العناصر الموجودة في الطابور ويبدأ من الموقع الذي يؤشر عليه (Front) إلى الموقع الذي يؤشر عليه (Rear) وبزيادة بمقدار واحد لحين الوصول إلى نهاية الطابور.

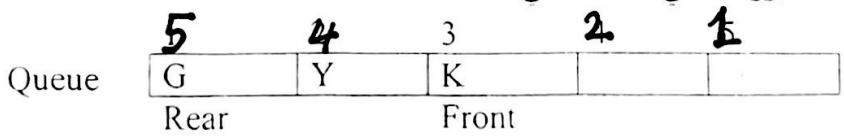
- **البرنامج الفرعي لتصفير محتويات الطابور:**

```
void clear()
{
front=-1;
rear=-1;
}
```

مما سبق لاحظنا في المثال الاول من هذا الفصل انه من مساوئ الطابور قد تكون هناك خلايا (مواقع) فارغة في مقدمة الطابور ولا يمكن استغلالها لان المؤشر (Rear) المسول عن عملية الإضافة يكون على الموقع الأخير ولحل هذه المشكلة وجد الطابور الدائري (**Circular Queue**).

3. 3 الطابور الدائري Circular Queue

عند اضافة عنصر إلى الطابور تتطلب هذه العملية تدقيق قيمة المؤشر (Rear) عن طريق استخدام هذه العبارة (Rear == Front) اذ ان ذلك يعني ان الطابور مملوء حتى وان كانت هنالك مواقع خالية في مقدمة الطابور كما في الشكل الاتي:



اي إننا في هذه الحالة سوف نخسر مساحة خزنيه ولتجنب هذه الحالة نستطيع استخدام الطابور استخداما دائريا وذلك بان نسمح للمؤشر (Rear)

- البرنامج الفرعي لعملية إضافة عنصر (Insert) إلى الطابور الدائري:

```
void insert()
{
if (((rear==size)&&(front==0)) || (front==rear+1))
cout<<"FULL QUEUE"<<endl;
else
if (rear==size) rear=0;
else
{
cout<<"Enter the Element : ";
cin>>element;
Queue[rear]=element;
rear=rear+1;
if (front==0) front=1;
}
}
```

- الفرعي لعملية حذف عنصر (Delete) من الطابور الدائري:

```
voiddelet()
{
if (front<=0) cout<<"EMPTY Queue "<<endl;
else
{
cout<<Queue[front-1]<<endl;
Queue[front-1]=0;
if (front==rear) {front=0; rear=0;}
else
if (front==size) front=1;
else front=front+1;
}
}
```


Scanned with CamScanner